REPUBLIQUE DU CAMEROUN
Paix – Travail – Patrie

MINISTERE DE
L'ENSEIGNEMENT SUPERIEUR

REPUBLIC OF CAMEROON
Peace – Work – Fatherland

MINISTRY OF HIGHER
EDUCATION

UNIVERSITY OF BUEA

FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER ENGINEERING

REPORT ON THE ANALYSIS AND DESIGN OF A HOSPITAL MANAGEMENT SYSTEM

Course: CEF331 – Object Oriented Modelling and UML
Course Instructor: Dr. Djouela Ines
Project by: Group 1

Group Members

Name	Matriculation Number
LORNA GLORIA NGWESAH NCHITUH	FE24A302
HAMASALI ALIM	FE24A289
ANYANGWE AKWI OKAWA	FE24A216
NKEMNKENG BLESSING NTSAFAC	FE24A346
TAMOKOUE FOGUE LANDRY BERTHOLD	FE24A625
RIGHTEOUSNESS NTONWI AJUYO	FE24A361
EYONG NGOE WILLY KERTIS	FE24A274
NDZO NGUM JOACHIM	FE24A331
ATEMKENG PRINCE LOVET TANZE FONCHENALLA	FE24A230
WEMNJE CALEB MBANJI WEPNYU	FE24A388
CHO SUCCESS MBO	FE24A245

Table of Contents

Group Members....2
EXECUTIVE SUMMARY....4
INTRODUCTION....5
Project Definition....5
SDLC Model Used....5
Functionality of the system....7
MODELING THE SYSTEM....9
USE CASE DIAGRAM....9
Actors and Use cases....9
Textual Description of Use cases....9
Use case diagram....22
CLASS DIAGRAM....23
Classes and relationships involved....23
Class diagram....24
SEQUENCE DIAGRAM....27
ACTIVITY DIAGRAM....31
CONCLUSION....33

EXECUTIVE SUMMARY

The purpose of this report is to document the steps taken in the analysis and design of a hospital management system (HMS). Such a system is intended to automate core operations that are carried out in a hospital, including patient registration, scheduling of appointments, clinical consultations, laboratory services, pharmacy management, billings and much more. The system is designed to improve efficiency and data accuracy within a healthcare facility.

An agile software development life cycle (SDLC) methodology was selected to support the dynamic and evolving nature of healthcare requirements. This approach enables incremental development, continuous stakeholder involvement, and early identification of risks so as to ensure that the system remains adaptable to changing requirements and user feedback while maintaining high reliability and patient safety standards.

The design is modeled using the Unified Modeling Language (UML) to provide a clear and well-structured representation of the functionalities of the system, and the various interactions with external actors.

- Use case diagrams identify key actors like patients, doctors, nurses, pharmacists, lab technicians, receptionists, and administrators. They identify and describe their interactions with the system. Each use case is supported by a detailed textual description.
- Class diagrams define the core entities or objects of the system and their relationships, establishing a solid object-oriented structure for managing patient records, appointments, prescriptions and billing information.
- Sequence diagrams and activity diagrams illustrate the flow of each interaction across major operational scenarios of the system such as patient treatment, and billing processes.

Overall, the proposed HMS offers a scalable, secure, and maintainable solution that enhances healthcare services, reduces manual errors and supports confidentiality in the health management system through well-structured system analysis and design.

INTRODUCTION

Project Definition

In healthcare institutions such as hospitals and health centers, proper coordination of activities and records of health information is greatly required to provide services for saving lives and medical advice within required times. Traditionally, manual methods have been used for this purpose and problems of time consumption, human errors and inconsistency have sometimes brought catastrophic outcomes within the healthcare milieu.

The Hospital Management System under design here is a proposed solution to the problem mentioned above: a software tool that can be used to streamline major activities of a healthcare facility. It addresses these challenges by providing a centralized, automated and integrated platform that supports the complete patient life cycle (registration → appointment scheduling → diagnosis → treatment → prescriptions of medications → billing and reporting) thereby improving data consistency, workflow coordination, and overall service efficiency.

SDLC Model Used

In order to support the dynamic and evolving nature of healthcare requirements, the Agile Software Development Life Cycle (SDLC) model is adopted for the development of the system. In a hospital setting, requirements frequently change due to evolving clinical procedures, regulatory policies, staffing structures and user feedback from medical personnel. For example, appointment workflows may require adjustment to reduce patients waiting time, or billing processes may need modification to accommodate insurance policies. The agile model supports such scenarios by the following means:

- **Early delivery of core functionalities:** Core hospital operations (such as patient records, appointments, and staff access) can be deployed early, allowing the system to support real operations while remaining functionalities are under development.
- **Improved Requirement Validation:** Medical and administrative stakeholders can review working features at the end of each iteration, reducing the risk of misinterpreting operational requirements or clinical workflows.
- **Reduced Development Risk:** By breaking the system into manageable increments, technical and functional risks are identified early. Faults in one module (e.g billing) do not compromise the entire system.
- **Easier Accommodation of Change:** Hospital policies, treatment procedures, and data handling requirements frequently change. This model allows such changes to be introduced in later iterations of development without need for much readjustments.

- **Continuous Testing and Quality Assurance:** Each increment undergoes testing before integration, which is particularly important for this system handling sensitive medical data where reliability and correctness are mandatory.
- **Better System Scalability and Maintainability:** The modular nature of incremental development results in a well-structured system that is easier to extend, maintain, and audit over time.

All these characteristics of the agile framework ensure that critical functionalities of the HMS such as patient records and clinical workflows can be validated early and refined continuously without disrupting the hospital operations.

Several SDLC models were evaluated during the design of the Hospital Management System. However, these models were not adopted due to limitations that make them less suitable for the dynamic and mission-critical nature of healthcare systems.

Waterfall Model

The Waterfall model follows a rigid, sequential development approach where each phase must be completed before the next begins. This model was not selected because hospital systems often experience frequent requirement changes, such as policy updates, workflow adjustments, and regulatory demands. The lack of flexibility and late-stage testing in the Waterfall model increases the risk of costly redesigns and delayed error detection.

V-Model

Although the V-Model improves upon Waterfall by emphasizing validation and verification, it still relies on early and fixed requirement definitions. In a hospital environment where user needs evolve through system use and feedback from medical staff, this rigidity limits adaptability and slows response to real-world operational challenges.

Spiral Model

The Spiral model is well-suited for large, high-risk projects due to its focus on risk analysis. However, it was not adopted because it is complex, resource-intensive, and costly to manage. For a hospital management system with clearly defined functional modules, the overhead associated with continuous risk assessment was considered unnecessary.

Incremental Model

The Incremental model allows the system to be developed in parts; however, it lacks the continuous user feedback and adaptive planning provided by Agile. Incremental development without frequent stakeholder involvement may lead to misalignment with actual hospital workflows.

Prototyping Model

While prototyping helps clarify user requirements, it often results in poor documentation and unrealistic user expectations if not carefully controlled. Given the need for reliability, maintainability, and compliance in healthcare systems, a prototype-driven approach alone was deemed to be insufficient.

Functionality of the system

The proposed HMS supports key functionalities including patient and staff management, appointment scheduling, electronic medical records, prescriptions handling, laboratory and pharmacy coordination, billings and payment processing, role-based access control and administrative reporting. It is a centralized information system within the hospital's local network.

The platform is accessible through a dedicated desktop application within the hospital's intranet, allowing authorized users to interact with the system based on their assigned roles. Hospital staff can use it to manage patient information and operational workflows, depending on their role in the hospital. Patients interact with the platform indirectly through hospital staff or directly through designated interfaces for appointment scheduling and information access, where applicable.

Core hospital processes are executed withing the platform. Each operation is recorded in real time, ensuring data consistency, traceability and secure access control across all departments within the hospital.

The platform enforces role-based access control, ensuring that users can only access functionalities and data relevant to them. Through structured system design and iterative development, the HMS enhances efficiency in operations and reduces the chances of errors in the hospital environment.

MODELING THE SYSTEM

The Unified Modeling Language (UML) is used to design the system's structure and workflow of various scenarios and use cases. UML represents the system's functional structure, behavior and interaction flows in a visual and logical manner.

The modeling process captures how users interact with the system, how system components are organized, and how data flows through various operations. By using diagrams such as class, use case, activity and sequence diagrams, the system is examined from all required perspectives, ensuring functional correctness and accuracy in the architecture.

Use case diagrams identify system actors and define the core services provided by the system. These are accompanied by textual descriptions that clarify the purpose, exceptions and outcomes of each use case. Class diagrams are then used to show the static structure of the system, illustrating classes, attributes, methods and the relationships between system entities.

To further describe system behavior, sequence diagrams are provided for selected scenarios, showing the chronological interaction between objects during system execution. Finally, activity diagrams model the flow of control and decision-making within critical processes of the system.

Together, these diagrams provide a comprehensive and coherent representation of the system, and they serve as a foundation for implementation, testing and future enhancement of the system.

USE CASE DIAGRAM

Actors and Use cases

The use case diagram organizes the needs of the system, identifies and specifies the major functionality that the system will offer to users. It shows the actors on the system and relationships between functionalities such as dependencies, inclusions and relationships.

The actors of the system are: Patient, Doctor, Nurse, Receptionist, Pharmacist, Laboratory Technician, System Administrator, System. Use cases include Register Patient, Schedule Appointment, View Medical Records, Diagnose Patient, Prescribe Medication, Conduct Laboratory Test, Manage Billing, Dispense Medication, Manage Users, Generate Reports.

Textual Description of Use cases

1. **Register Patient**

   Actor: Receptionist

   Description: A process to record new patient details or verify existing ones to initiate a medical consultation.

Precondition: The patient is present and provides identification, the Receptionist is authenticated and logged into the hospital system.

Postcondition: A unique Patient ID is generated (for new patients) or retrieved (for existing patients). The patient is added to the consultation queue and medical staff are notified

Main flow:

- The Patient provides their Name or ID to the Receptionist.
- The Receptionist inputs the ID into the system search bar.
- The System queries the Database to check if a matching record exists.
- The Database returns a "No Record Found" status.
- The System opens a blank registration form.
- The Receptionist enters the patient's personal and contact information.
- The System performs an internal validation check on mandatory fields.
- The System sends a command to the Database to save the new record.
- The Database confirms the save and returns a unique, newly generated Patient ID.
- The System prompts the Receptionist to link the registration to a medical consultation.
- The Receptionist selects and assigns the patient to an available Nurse or Doctor.
- The System sends an automated notification to the assigned medical staff.

Alternate flow: If an existing record is found, the system displays the existing profile.

If the system detects a duplicate during the entry, the Receptionist updates the existing profile instead of creating a new one.

Exception: Missing data, no medical staff available.

Primary Actor: Receptionist / Patient

Description: This use case enables the scheduling of an appointment between a patient and a doctor based on availability.

Precondition: The patient is registered in the system

Postcondition: An appointment record is created with a "Scheduled" status

2. **Check-in Patient**

   Actor: Doctor

   Description: Recording the arrival of a returning patient for a scheduled appointment.

   Precondition: Patient is already registered in the system; a scheduled appointment exists for the day.

   Postcondition: Appointment status is updated to "Arrived" and the Nurse is notified.

   Main flow:

   - Receptionist searches for the patient by Name or ID.
   - System displays the scheduled appointment for the current time.
   - Receptionist confirms the patient's arrival.
   - System alerts the Nurse that the patient is ready for vitals and consultation.

   Alternate flow: None

   Exception: If no appointment is found, the system redirects the Receptionist to the scheduling module or the walk-in registration flow.

3. **Schedule appointment**

   Actor: Receptionist, Patient, Doctor

   Description: Set a slot for meeting with the doctor

   Precondition: Doctor is available

   Postcondition: Appointment is scheduled

   Main flow:

   - Select doctor and time
   - System checks availability
   - Appointment is scheduled

   Alternate flow: No slot available, suggest alternative doctors

   Exception: Conflicting schedules.

   Exception flow: None

4. **Manage Doctor scheduling**

   Primary Actor: System administrator

   Description: This use case allows the System Administrator to create, modify and maintain doctors' work schedules withing the HMS. It ensures that doctors are

appropriately assigned to shifts, clinics, and on-call duties based on availability, specialization, and hospital operational requirements, thereby preventing conflicts with schedules and service gaps.

Precondition: The doctor is registered into the system

Postcondition: Doctor schedules are created or updated. Schedule changes are stored and maid available to relevant hospital staff.

Main flow:

- The System Administrator accesses the scheduling module.
- The system displays existing doctor schedules.
- The administrator selects a doctor or department
- Shift times, duty periods, or on-call assignments are defined and updated.
- The updated schedule is saved and confirmed.=

Alternative flow: Emergency adjustment: The administrator assigns temporary changes due to emergencies or staff shortages.

Exception flow: Doctor not available during selected time period. Scheduling conflicts that cannot be resolved automatically. System access or permission error.

5. **Schedule nurse shifts**

Primary Actor: System Administrator

Description: This use case enables the System Administrator to create, modify and manage work shifts for nurses in the HMS. It ensures adequate nurse scheduling across all hospital departments and wards while preventing scheduling conflicts and supporting efficiency in hospital operations.

6. **Use Case: Allocate Beds or Theatre**

Primary Actor: Doctor

Secondary Actor: System Administrator

Description: This use case assigns hospital beds or operation theatres and admission wards.

Precondition: The Patient has been recommended for admission by the Doctor and resources are available.

Postcondition: Resource is allocated to the Patient

Main flow:

- Doctor selects required resources (ward, bed etc)
- System Admin checks availability
- Allocation is confirmed
- The system stores the results and updates the Patient's medical record.

Alternative flow: Emergency override

Exception: No available beds or wards

7. **Record Vitals**

Primary Actor: Nurse

Description: Capture patient's vital signals like blood pressure, temperature and such.

Precondition: Patient is checked-in.

Postcondition: Vitals stored in the medical record.

Main flow:

- The vitals are measured
- The readings are done and entered into the system
- The data is saved into the system.

Alternative flow: None

Exception flow: Instrument malfunction

8. **Manage medical equipment**

Primary Actor: System Administrator

Description: This use case allows the System administrator to track and maintain equipments of the hospital.

Precondition: Equipment is registered.

Postcondition: The status of the equipment is updated.

Main flow:

- The System Administrator updates the equipment's status.
- Changes are saved

Alternative flow: Equipment damaged, then the System Administrator makes provision for repairs and updates the status as "In Maintenance".

Exception: Data inconsistency or the equipment is damaged beyond repair.

9. **Manage medical history**

Primary actor: Doctor

Secondary actors: Nurse (optional), System Administrator (audit and access control)

Description: This use case allows medical personnel to create, view, and update a patient's medical history within the HMS. It ensures that diagnoses, treatments, lab results and prescriptions are accurately recorded and maintained to support continuity of care and informed clinical decision-making;

Preconditions: The patient is registered in the system. A medical record exists for the patient. The actor is authenticated and has appropriate access permissions.

Postcondition: The patient's medical history is updated and securely stored. Changes are immediately available to authorized health personnel An audit of trails is maintained.

Main flow:

- The doctor selects a patient from the system.
- The system retrieves the patient's medical record.
- The doctor views existing medical history entries.
- The doctor adds or updates diagnoses, treatment notes, or prescriptions.
- The system validates the information entered.
- The updated medical history is saved and confirmed.

Alternate flow: If the actor has read-only permissions, the system allows viewing of medical record without modification. Nurses can also record vitals which are added to the medical history

Exception: Patient record not found. Insufficient access permissions. System error in saving data

10. **Order lab test**

Actor: Doctor

Description: This use case is where the doctor requests for diagnostic tests from the laboratory

Precondition: Patient is receiving medical care.

Postcondition: Lab request created

Main flow:

- Select test
- Submit order

Alternative flow: Urgent test indicated

Exception: Invalid test selection

11. **Receive and process sample**

Actor: Lab Technician

Description: Lab Technician processes collected samples

Precondition: Lab test ordered by doctor

Postcondition: Sample processed.

Main flow:

- Receive sample
- Perform test
- Record findings.

Alternative flow: Samples are re-collected

Exceptions: Sample contamination.

12. **Enter test results**

Actor: Lab Technician

Description: This use case enables the Lab Technician to upload the lab test outcomes into the system.

Precondition: Lab Test confirmed

Postcondition: Results are made available to the doctor

Main flow:

- The Lab technician enters the results
- The results are saved to the system

Alternative flow: Results are amended.

Exception: Data entry error.

13. **Prescribe medication**

Actor: Doctor

Description: This use case allows the doctor to issue a medication prescription to the patient.

Precondition: Diagnosis is confirmed.

Postcondition: Prescription created

Main flow:

- Select medication
- Define dosage
- Save prescription

Alternative flow: Substitute medication

Exception: Allergy indications.

14. **Dispense medication**

Actor: Pharmacist

Description: This use case allows the pharmacist to provide the prescribed drugs to the patient.

Precondition: Valid prescription from the doctor exists.

Postcondition: Medication is dispensed.

Main flow:

- Verify prescription
- Dispense drugs

Alternative flow: Partial availability of the prescribed drug, or no availability at all then the pharmacist substitutes with an appropriate alternative.

Exception: Expired prescriptions, unavailable options or allergy indications.

15. **Update discharge summary**

Primary actor: Doctor

Secondary actor: Nurse

Description: This use case enables the doctor to update and finalize the discharge summary of a patient after treatment or hospitalization. It shows key clinical information such as diagnosis, treatments provided, medications prescribed, follow-up instructions and patient's condition at discharge.

Precondition: The patient is registered in the system. The patient has an active or complete admission record. The doctor is authenticated and authorized to update the medical record.

Postcondition: The patient's discharge summary is updated and stored in the medical record and a copy is made available to them. The summary is also made available to relevant hospital departments. The patient's status is marked as discharged

Main flow:

- The doctor selects the patient who is ready for discharged
- The system retrieves the patient's medical record
- The doctor reviews diagnosis, treatment history, and test results.
- The doctor enters or updates discharge notes and follow-up instructions.
- The system validates the entered information
- The discharge summary is saved, confirmed, and a copy made available to the patient

Alternative flow: The doctor saves the discharge summary as a draft for later completion.

Exceptions: Patient record not found. Discharge not permitted due to pending medical procedures or unpaid bills. Insufficient access permission to grant discharge. System errors

16. **Process payment**

Primary actor: Receptionist

Secondary actor: Patient, System Administrator and Finance Department

Description: This use case enables the processing of payment for hospital services rendered to the patient, It ensures that all charges are settled, payment status recorded and financial records are updated accordingly within the HMS.

Precondition: The patient is registered in the system. An invoice has been generated for the patient. The receptionist is authenticated and authorized.

Postcondition: The payment is successfully recorded in the system. The invoice payment status is updated accordingly. A payment confirmation receipt is generated.

Main flow:

- The receptionist selects the patient's invoice
- The system displays invoice details and the amount due for payment

- The patient provides payment details
- The system validates the payment information
- The payment is processed and confirmed.
- The system updates the invoice payment and records the transaction.

Alternative flow: Patient might make a partial payment. Payment can be done online through integrated payment mediums.

Exception: Invalid or incomplete payment information. Payment failure due to insufficient funds or network errors. Invoice not found or already settled. System errors due to payment processing

17. **Generate final bill**

Primary actor: Receptionist

Secondary actors: System administrator and Finance Department

Description: This use case enables the generation of a final bill for a patient after all medical services have been rendered.

Precondition: The patient is registered in the system. All chargeable services to the patient have been recorded. The receptionist is authenticated and authorized.

Postcondition: A final bill is generated and linked the patient's invoice. The total payable amount is calculated and stored. The bill is made available for payment processing.

Main flow:

- The receptionist selects the patient for billing
- The system retrieves all services rendered to the patient and their associated charges
- The system calculates the total payable amount
- The receptionist reviews the generated bill
- The system generates the final bill and assigns a unique invoice identification
- The final bill is saved and confirmed

Alternative flow: The system takes into consideration insurance deductions and grants, then recalculates the payable amount where applicable.

Exception: Missing or incomplete service records. Billing calculation errors. Unauthorized attempt to generate a bill. System errors during bill generation.

18. **Generate financial report**

Primary actors: System Administrator and Finance Department

Description: this use case enables authorized administrative and financial personnel to generate financial reports summarizing hospital revenue, expenses, and payment records over a specified period.

Precondition: Financial transactions (invoices and payments) exist in the system. The actor is authenticated and authorized to access financial data.

Postcondition: A financial report is generated based on selected criteria. The report is stored and made available for viewing or exporting.

Main flow:

- The actor accesses the financial reporting module.
- The system displays available reporting options.
- The actor selects report parameters (such as date range, department, payment status)
- The system retrieves the relevant financial data.
- The system generates the financial report
- The report is displayed and confirmed.

Alternative flows: None.

Exception: No financial data available for the selected criteria or period. Insufficient access permissions. System errors during report generation.

19. **Raise reorder request**

Primary actor: Pharmacist

Secondary actor: System administrator

Description: This use case allows the pharmacist to request a reorder for medications when stock levels fall below predefined thresholds. This is to avoid shortage of stock that could negatively affect patient treatments.

Precondition: The pharmacist is authenticated. Medication inventory record exists in the system. Reorder threshold levels are defined.

Postcondition: A reorder request is created and recorded in the system. The request is forwarded to the System Administrator for approval. Inventory status is updated to reflect the pending reorder.

Main flow:

- The pharmacist accesses the inventory management module.
- The system displays current medication stock levels
- The pharmacist selects a medication required replenishment or restocking.
- The pharmacist enters reorder quantity and justification.
- The system validates the reorder requests
- The reorder request is submitted to the System Administrator and confirmed.

Alternative flows: The system automatically indicates medications below threshold levels and suggests reorder requests. The pharmacist submits requests for multiple medications.

Exceptions: Invalid reorder quantity. Insufficient access permissions. System errors.

20. **Protect Data/Manage user access**

Primary Actor: System Administrator

Description: This use case enables the System Administrator to manage user access rights and to enforce data protection measures within the HMS, preventing unauthorized access and making sure all other actors have access to the information they should.

Precondition: The System Administrator is authenticated and authorized. User account exists in the system. Security policies and access roles are defined.

Postcondition: User access permissions are created, so that that they can view their information on the HMS or do some modifications (depending on their roles). Data protection rules are enforced across the system.

Main flow:

- The System Administrator accesses the user management module
- The system displays a list of users and their assigned roles
- The administrator selects a user or role to manage
- Access permissions are assigned, modified, or revoked.
- The system applies data protection rules based on updated permissions
- The system saves the configurations.

Alternative flow: The Administrator can grant access for temporary tasks or emergencies.

Exceptions: Unauthorized attempt to modify access rights. Conflicts between assigned permissions and security policies

21. **Allow patient access**

Primary Actor: System Administrator

Description: This use case enables the System Administrator to grant, modify or remove patient access to the HMS. It allows patients to securely view their permitted information such as appointments, medical summaries, prescriptions and more, while ensuring data privacy and controlled access.

Precondition: The patient is registered on the system. A patient user account exists or is being created. The System Administrator is authenticated and authorized.

Postcondition: Patient access credentials are created, updated or revoked. The patient is able to securely access authorized system features.

Main flow

- The System Administrator accesses the user access management module.

  The System displays existing patient accounts and access statuses.
- The Administrator selects a patient account or creates a new one
- Access rights are modified or granted
- The system saves the access configuration and confirms the update.

Alternative flow: None

Exceptions: Patient record not found. Unauthorized attempt to modify patient access. Conflict with data protection policies. System errors during access configuration.

22. **Manage referrals to External Facilities**

Primary Actor: Doctor

Secondary actor: Receptionist, System Administrator

Description: This use case supports coordinated healthcare delivery by managing referrals to external medical facilities and specialties, promoting structured operations of the hospital.

Precondition: The patient is registered on the system. The doctor is authenticated and authorized. External facilities are predefined in the system.

Postcondition: The referral status is recorded for follow-up purposes.

Main flow:

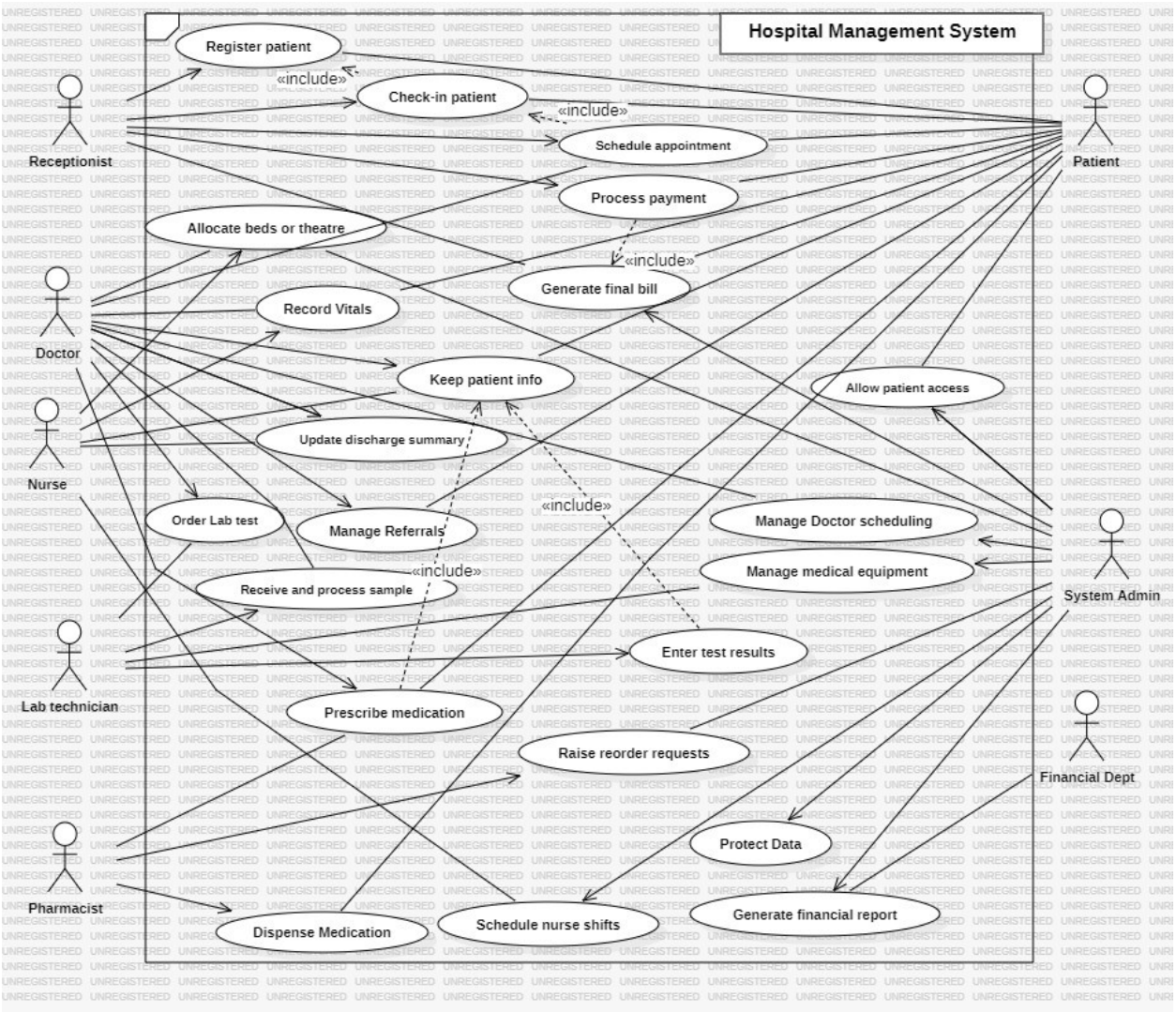
- The doctor selects a patient requiring external referral
- The system retrieves the patient's medical record
- The doctor selects an external facility or specialist
- The doctor enters referral details and justification
- The system validates the information
- The referral record is saved and confirmed

Alternative flows: If the required facility is not listed, the Doctor requests the System Administrator to add it to the system.

Exceptions flow: Patient record not found. Insufficient access permissions. System error during referral creation.

Use case diagram

Below is the use case diagram for the system showing actors and use cases of the system.



CLASS DIAGRAM

Classes and relationships involved

The class diagram defines the structure of the system and provides a blueprint for objects, showing their methods, attributes and relationships between them. Classes included here are:

Doctor, Patient, Appointment, MedicalRecord, Receptionist, Nurse, Prescription, Bill. LabOrder, LabStaff, invoice, Pharmacist, Invoice, SystemAdministratorAndFinanceDept

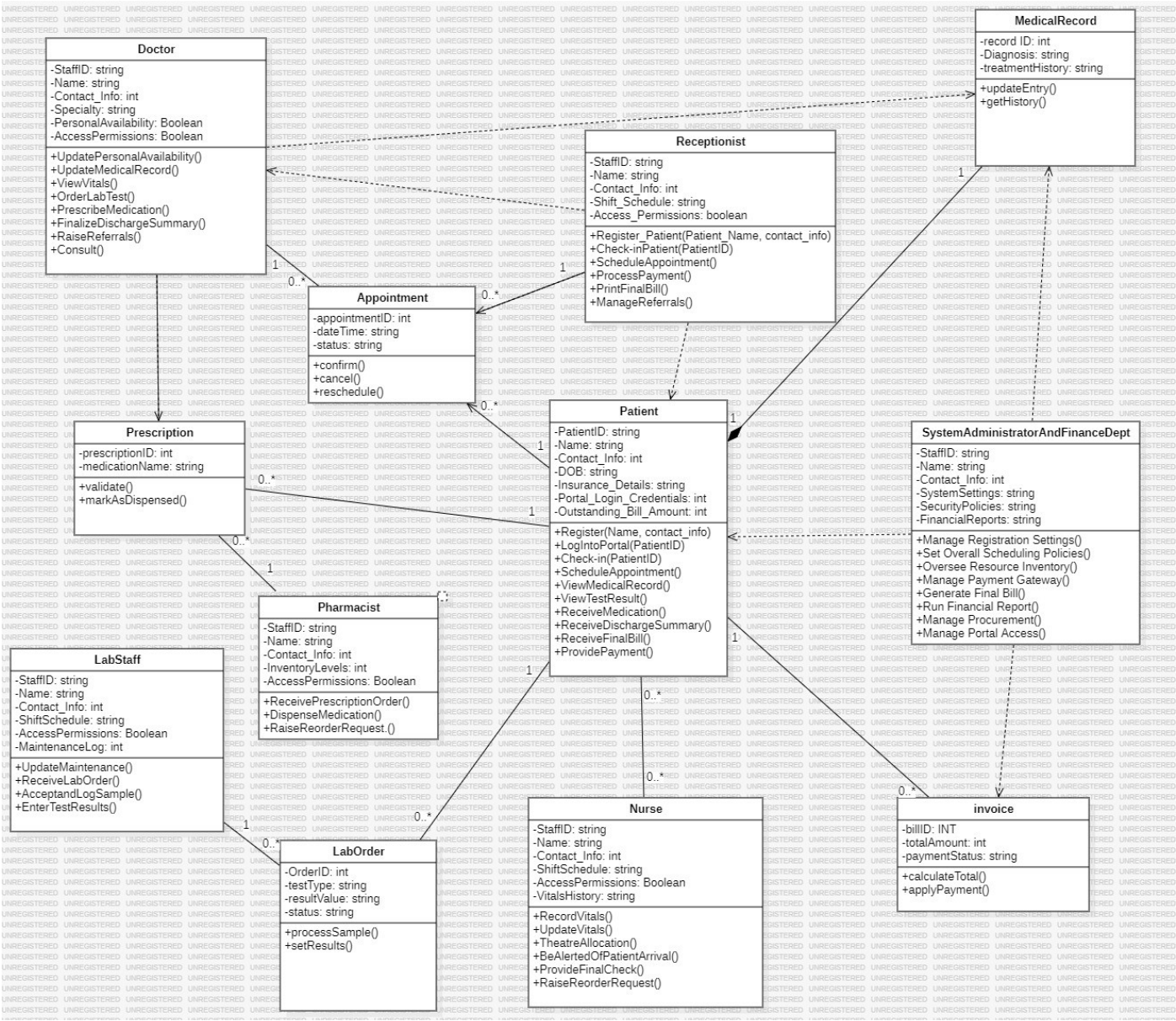
These are some relationships between these classes:

1. There is an association between Doctor and Patient. One Doctor can treat many patient. A patient is treated by a doctor or multiple doctors during consultation.
2. There is an association between Doctor and Appointment. A Doctor can have many appointments and each Appointment is assigned to exactly one doctor.
3. There is an association between Patient and Appointment. One Patient can have multiple appointments. Each appointment belongs to one patient.
4. There is an association between Receptionist and Patient. One Receptionist can register and manage many patients within the system.
5. There is an association between Receptionist and Appointment. One Receptionist can schedule, modify, or cancel many appointments.
6. There is an aggregation relationship between Patient and MedicalRecord. One Patient has one medical record. A medical record is associated with a patient but can exist independently for historical and legal purposes
7. There is an association between Doctor and MedicalRecord. One Doctor can view and update many medical records during patient care.
8. There is an association between Doctor and Prescription. One Doctor can issue multiple prescriptions. Each prescription is issued by one doctor.
9. There is an association between Patient and Prescription. One Patient can receive many prescriptions over time. Each prescription belongs to a patient.
10. There is an association between Pharmacist and Prescription. One Pharmacist can dispense many prescriptions after validation.
11. There is an association between Pharmacist and Patient. One Pharmacist can dispense medication to many patients. A patient may receive medication from different pharmacists over time.

12. There is an association between LabStaff and LabOrder. One laboratory staff member can process many lab orders. Each lab order is handled by authorized laboratory staff.
13. There is an association between LabStaff and LabOrder. One laboratory staff member can process many lab orders. Each lab order is handled by authorized laboratory staff.
14. There is an association between Patient and LabOrder. One Patient can have multiple lab orders. Each lab order belongs to a specific patient.
15. There is an association between Nurse and Patient. One Nurse can attend to many patients. A patient may be attended by multiple nurses during hospitalization.
16. There is an association between Nurse and LabOrder. A Nurse can assist in collecting samples for many lab orders.
17. There is an association between Patient and Invoice. One Patient can receive multiple invoices. Each invoice is generated for a specific patient.
18. There is an association between SystemAdministratorAndFinanceDept and Invoice. The system administrator or finance department can generate and manage many invoices.
19. There is an association between SystemAdministratorAndFinanceDept and Patient. The system administrator manages patient access, billing policies, and portal permissions for many patients.

Class diagram

Below is the class diagram for the system.



SEQUENCE DIAGRAM

Sequence diagrams are used to illustrate the dynamic behavior of the HMS by modeling

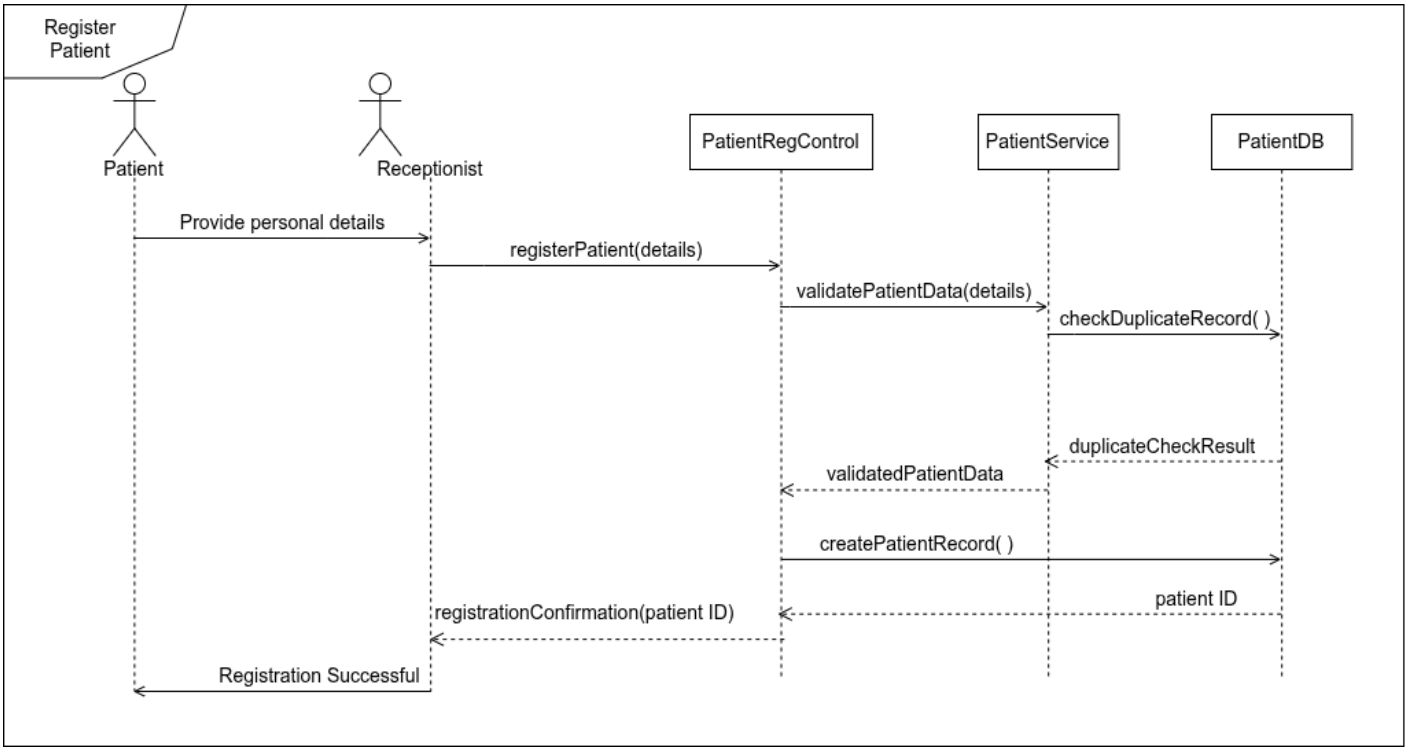
interactions between system components over time. While the class diagram defines the static structure of the system and the use case diagram identifies the the system's functionalities, sequence diagrams demonstrate how objects collaborate to execute critical system operations in a time-ordered manner.

The sequence diagrams presented in this section address several scenarios that represent the most critical and frequently executed workflows withing the hospital environment. These use cases were selected because they involve multiple actors, require coordination across several system components, and directly impact patient care, operational efficiency, financial processing.

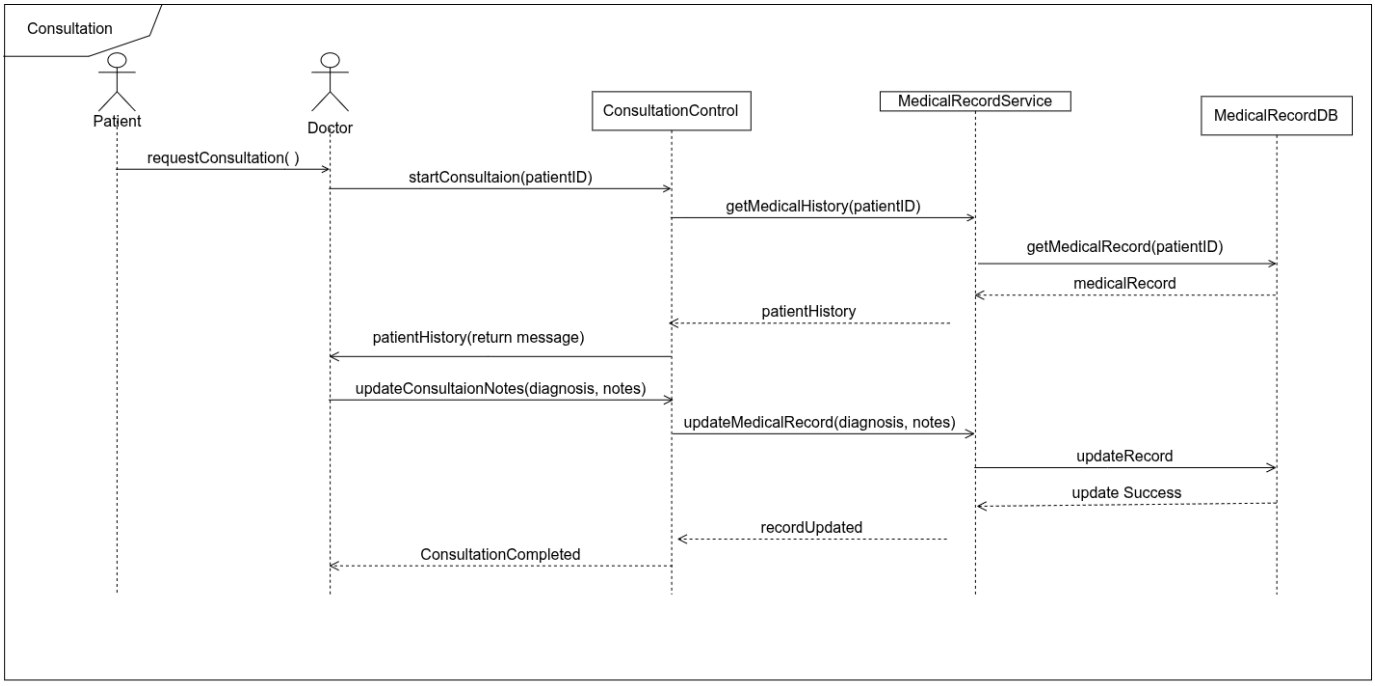
Each sequence diagram captures the interaction between users (such as patients and medical staff) and key system objects, including controllers, services, and data entities. The diagrams clearly show message flow, method invocations, decision points, and system responses, thereby providing a detailed view of how the system behaves during real-world hospital operations.

The following sequence diagrams are included

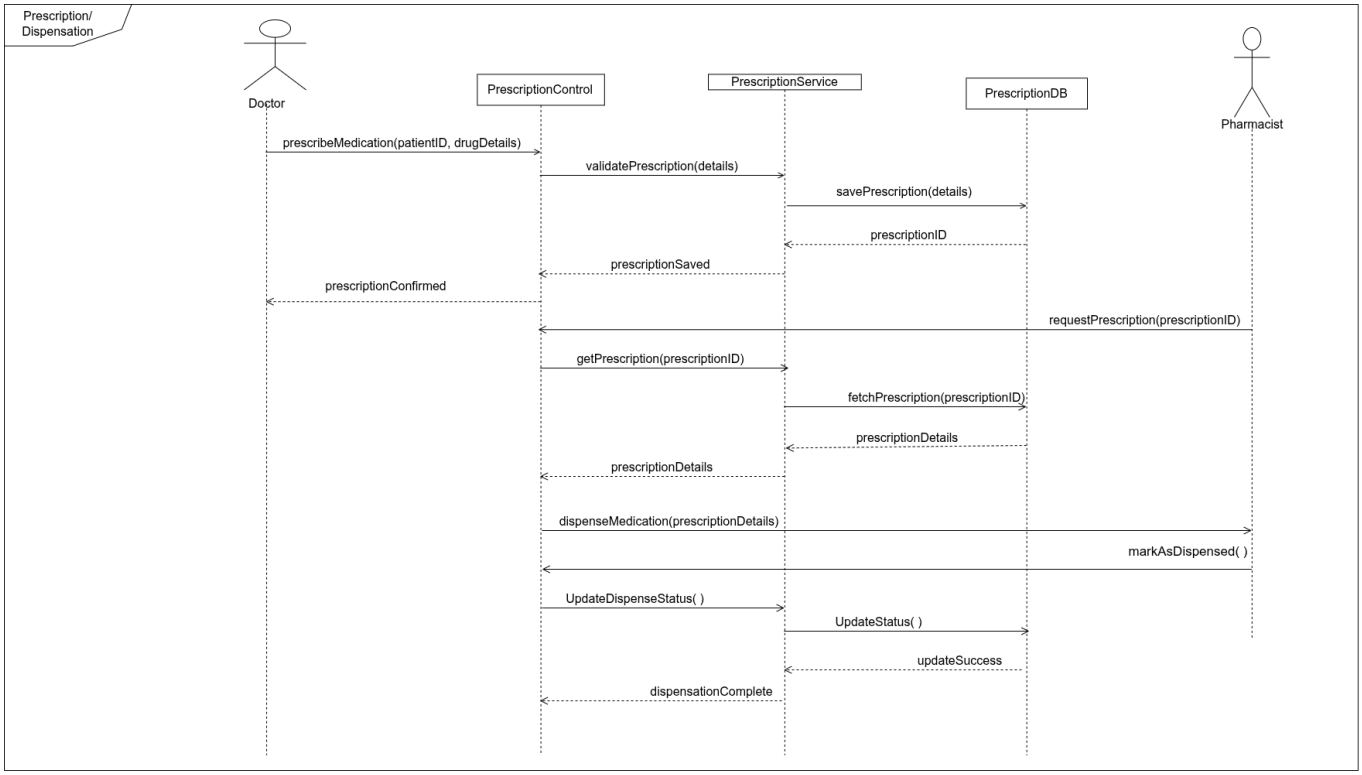
1. **Patient registration:** It illustrates the process of capturing patient details, validating information, and creating a patient record within the system.



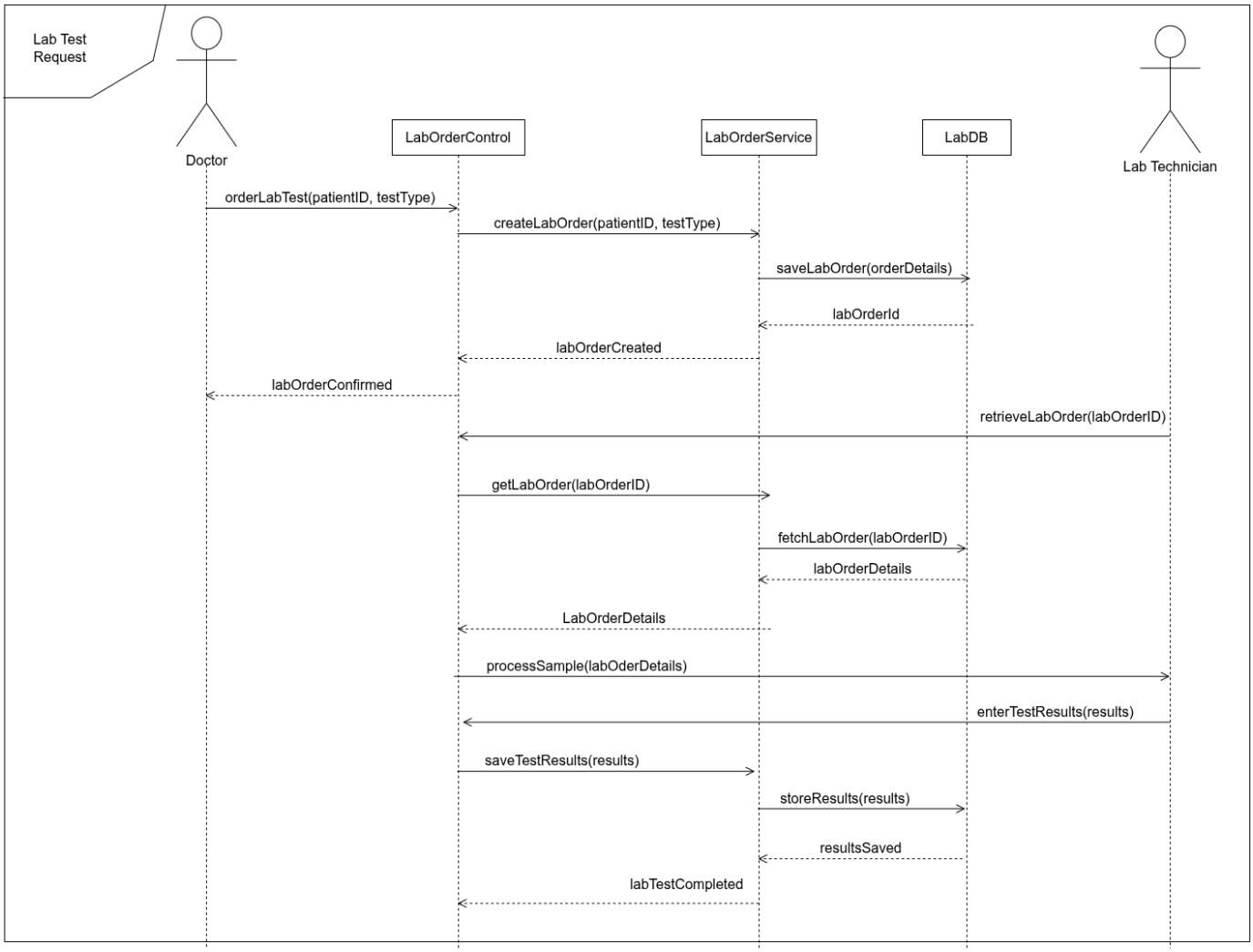
2. **Consultation:** It models the interaction between doctors, patients, medical records, and prescriptions during a clinical consultation.



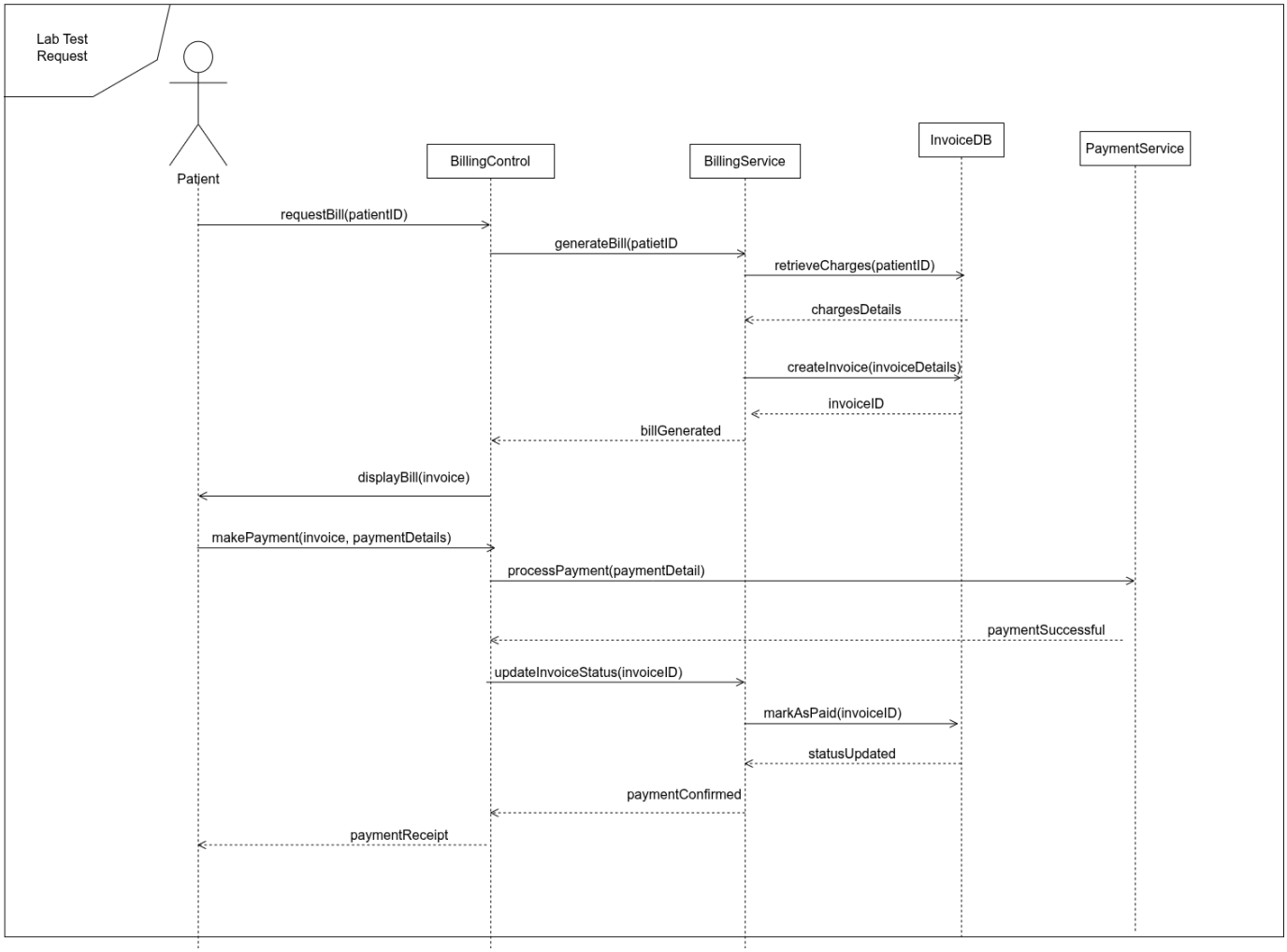
3. **Prescription and Medication Dispensing:** It represents the workflow for the process where the doctor creates a prescription on the system. Goes on to access and verification by the pharmacist who in turn dispenses the prescribed medication to the patient concerned.



4. **Laboratory Test request and Result Management:** It demonstrates how laboratory tests are requested, processed, and communicated back to medical staff.



5. **Billing and payment:** It addresses the process whereby the system generates an invoice based on the medical services offered to the patient, payment by the patient and validation/recording by the system.



Together, these sequence diagrams validate the correctness of the system design by showing how the proposed components interact to support essential hospital workflows. They also serve as a bridge between system requirements and implementation, ensuring that the system behavior aligns with real operational scenarios in a healthcare environment.

ACTIVITY DIAGRAM

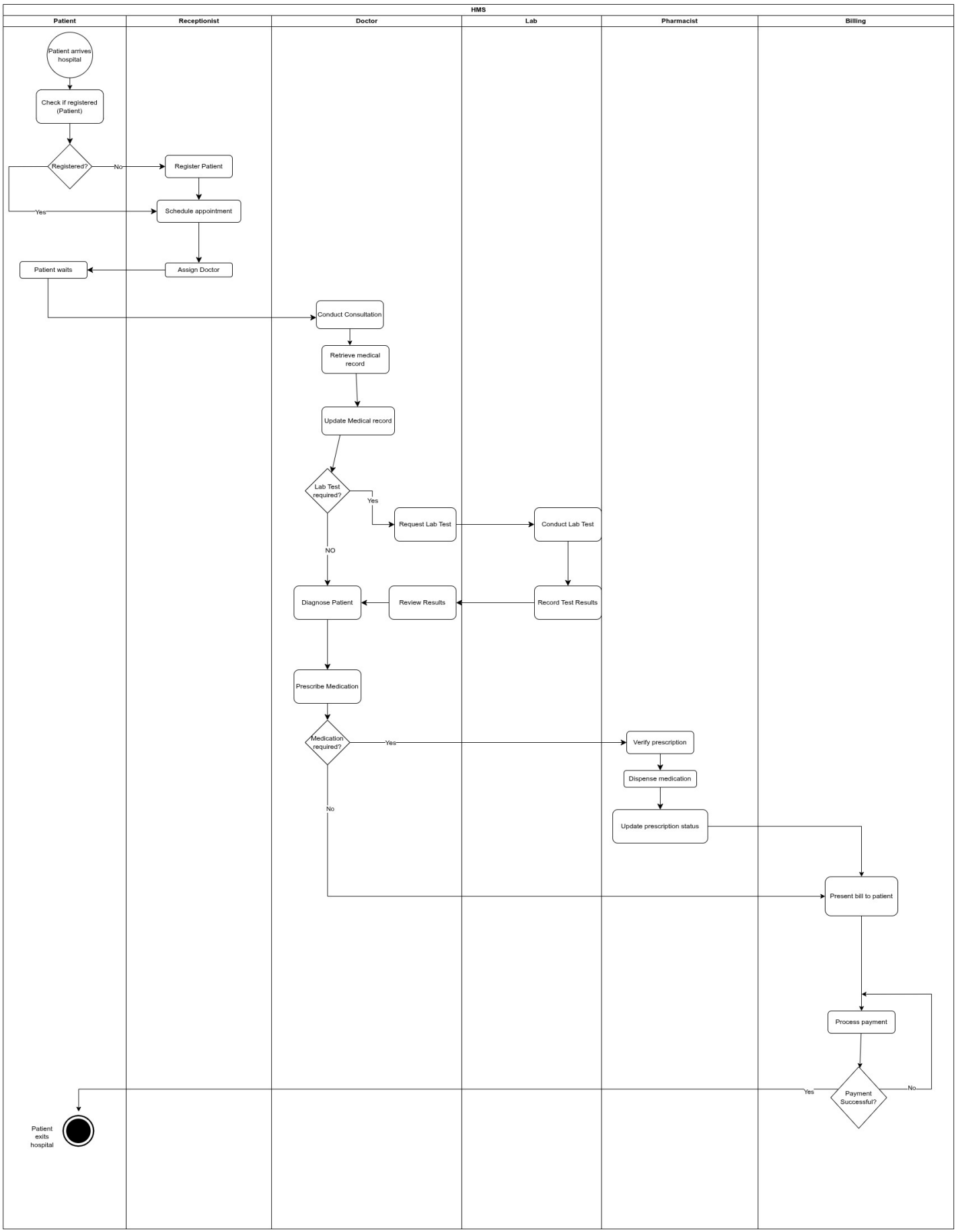
The activity diagram illustrates the overall operational workflow of the HMS from patient arrival to completion of service delivery. It provides a high-level view of how different hospital departments and actors interact through a sequence of coordinated activities to deliver healthcare services efficiently.

The process begins when a patient arrives at the hospital and initiates registration at the reception. At this stage, patient details are either newly recorded or retrieved from existing records within the system. Once registration is completed, the patient proceeds to consultation, where a doctor examines the patient and determines the appropriate course of action based on clinical assessment.

A decision point is introduced to determine whether laboratory tests are required. If laboratory investigation is necessary, a lab test request is generated and forwarded to the laboratory department, where the tests are performed and results are produced. These results are then returned to support accurate diagnosis. If no laboratory tests are required, the workflow bypasses this stage and proceeds directly to diagnosis.

Following diagnosis, the doctor prescribes appropriate medication, which is subsequently dispensed by the pharmacy. After medical services have been delivered, the system transitions to the billing phase, where charges are calculated based on consultations, laboratory tests, and medications provided. The patient then completes payment, and a receipt is issued to confirm successful transaction.

The activity diagram concludes when the patient exits the system, signifying the end of the service cycle. Overall, the diagram emphasizes clear sequencing, decision-making points, and coordination among hospital units, ensuring streamlined operations, reduced delays, and improved patient experience.


HMS
Patient
Receptionist
Doctor
Lab
Pharmacist
Billing
Patient arrives hospital
Check if registered (Patient)
Registered?
No
Yes
Register Patient
Schedule appointment
Assign Doctor
Patient waits
Conduct Consultation
Retrieve medical record
Update Medical record
Lab Test required?
Yes
NO
Request Lab Test
Conduct Lab Test
Record Test Results
Review Results
Diagnose Patient
Prescribe Medication
Medication required?
Yes
No
Verify prescription
Dispense medication
Update prescription status
Present bill to patient
Process payment
Payment Successful?
Yes
No
Patient exits hospital

CONCLUSION

This report presented the analysis, design, and modeling of a Hospital Management System aimed at improving the efficiency, accuracy, and coordination of healthcare service delivery. The study began by identifying key challenges in traditional hospital operations, including fragmented record management, inefficient scheduling, delays in service delivery, and limited data accessibility. In response to these challenges, a structured system design was proposed to streamline clinical, administrative, and financial processes within the hospital environment.

Using the Agile Software Development Life Cycle (SDLC) model, the system was designed to support iterative development, continuous stakeholder feedback, and adaptability to evolving hospital requirements. The choice of this model enables the system to accommodate future enhancements while ensuring early delivery of functional components and reduced development risk.

Unified Modeling Language (UML) diagrams were employed to clearly represent the system's structure and behavior. Use case diagrams and their corresponding textual descriptions captured user interactions and functional requirements, while class diagrams defined the system's static structure and relationships. Sequence diagrams illustrated the dynamic interactions for critical workflows, and the activity diagram provided an end-to-end view of hospital operations, ensuring consistency across all modeled components.

Overall, the proposed Hospital Management System demonstrates a comprehensive and scalable design capable of improving patient care, operational efficiency, data security, and decision-making within healthcare institutions. The system establishes a solid foundation for future extensions, such as advanced analytics, integration with external healthcare facilities, and enhanced patient self-service features. With proper implementation, the proposed solution has the potential to significantly modernize hospital operations and support sustainable healthcare delivery.